

C언어 1일차 과제 보고서

RO:BIT 수습단원 교육 · 과제 1 ~ 7

성명 : 최동현

학번 : 2026406040

개발 환경 : Visual Studio 2026

개요

본 보고서는 C언어 1일차 교육에서 다룬 변수·자료형, printf/scanf, 상수, 연산자, 조건문(if-switch), 반복문(for-while-do~while), break/continue 개념을 활용하여 과제 1번부터 7번까지를 구현한 내용을 정리한 것이다.

각 과제는 [문제 요약], [접근 방법], [핵심 코드], [사용한 개념], [예외처리 및 한계] 순서로 서술하였다. 모든 프로그램은 Visual Studio 2026 환경에서 빌드 및 실행을 확인하였으며, 소스 파일은 assignment1.c ~ assignment7.c 로 함께 제출한다.

입력 함수는 Visual Studio 환경 경고를 피하기 위해 파일 상단에 #define _CRT_SECURE_NO_WARNINGS 를 선언하고, scanf 를 사용하였다.

과제 1. 5개 실수의 평균·최댓값·최솟값

■ 문제 요약

5개의 실수를 입력받아 평균, 최댓값, 최솟값을 출력하는 프로그램을 작성한다.

■ 접근 방법

- 5개의 값을 다루기 위해 float형 배열(arr[5])을 사용하였다.
- 입력 반복문과 계산 반복문을 분리하여, 먼저 5개 값을 모두 입력받은 뒤 다시 순회하며 합·최댓값·최솟값을 구하도록 하였다.
- 배열 크기를 숫자로 직접 쓰지 않고 sizeof(arr)/sizeof(float) 로 계산해, 배열 크기가 바뀌어도 코드가 그대로 동작하도록 하였다.

■ 핵심 코드

```
float arr[5]; // 실수를 저장할 배열

for (int i = 0; i < sizeof(arr)/sizeof(float); i++) {
    printf("%d번째 실수를 입력하시오: ", i+1);
    scanf_s("%f", &arr[i]);
}

float mean = 0;
float max = arr[0]; // 초기값을 첫 번째 요소로 지정
float min = arr[0];
for (int j = 0; j < sizeof(arr)/sizeof(float); j++) {
    mean += arr[j];
    if (max <= arr[j]) max = arr[j];
    if (min >= arr[j]) min = arr[j];
}
printf("평균: %f\n", mean / (sizeof(arr)/sizeof(float)));
```

■ 사용한 개념

배열, for 반복문, sizeof 연산자, scanf_s, if 조건문, 산술 연산자(+, -, *, /), 관계 연산자(<, >).

■ 예외처리 및 한계

- 최댓값·최솟값 변수를 임의의 큰/작은 수가 아닌 arr[0] 으로 초기화하여, 입력값의 범위와 관계없이 항상 올바르게 비교되도록 하였다.
- 합계를 구한 뒤 배열 크기로 나눌 때에도 sizeof 계산식을 사용해 평균 계산의 분모가 배열 크기와 항상 일치하도록 하였다.
- 한계: 사용자가 실수가 아닌 문자를 입력하는 경우(scanf_s 실패)에 대한 별도 처리는 하지 않았다.

과제 2. N번째 피보나치 수

■ 문제 요약

n을 입력받아 n번째 피보나치 수를 출력한다. (1번째=0, 2번째=1, 이후는 앞 두 수의 합)

■ 접근 방법

- 1번째(0)와 2번째(1)는 규칙의 시작값이므로 조건문으로 먼저 예외처리하였다.
- 3번째부터는 앞의 두 값 p1, p2 를 이용해 next = p1 + p2 를 반복 계산하고, 계산 후 p1·p2 를 한 칸씩 갱신하였다.

■ 핵심 코드

```
scanf("%d", &num);
if (num == 1) printf("0"); // 1번째 예외처리
else if (num == 2) printf("1"); // 2번째 예외처리
else if (num >= 3) {
    int p1 = 0, p2 = 1, next;
    for (int i = 3; i <= num; i++) { // 0,1이 있으므로 3부터 시작
        next = p1 + p2;
        p1 = p2;
        p2 = next;
    }
    printf("%d ", next);
}
```

■ 사용한 개념

if / else if 조건문, for 반복문, 변수 갱신(대입 연산).

■ 예외처리 및 한계

- 피보나치 수열의 시작값인 1번째·2번째를 반복문에 넣지 않고 조건문으로 분리하여, 반복문이 3번째부터 자연스럽게 시작되도록 하였다.
- 한계: n이 0 이하로 입력되는 경우는 출력이 없으며, int 자료형을 사용하므로 n이 커지면(대략 47 이상) 오버플로우가 발생할 수 있다.

과제 3. 윤년 판별

■ 문제 요약

연도를 입력받아 윤년 여부를 판별한다. 윤년 조건은 '4의 배수이면서 100의 배수가 아니거나, 400의 배수'이다.

■ 접근 방법

- 먼저 4의 배수인지 확인하고, 그 안에서 (100의 배수가 아니거나 400의 배수) 조건을 논리 연산자 `||` 로 판별하는 중첩 조건문 구조를 사용하였다.

■ 핵심 코드

```
scanf("%d", &year);
if (year % 4 == 0) {
    if (year % 100 != 0 || year % 400 == 0)
        printf("윤년입니다.");
    else
        printf("윤년이 아닙니다.");
}
else {
    printf("윤년이 아닙니다.");
}
```

■ 사용한 개념

중첩 if / else 조건문, 나머지 연산자(%), 논리합 연산자(||), 관계 연산자(==, !=).

■ 예외처리 및 한계

- 400의 배수(예: 2000)는 윤년, 100의 배수이지만 400의 배수가 아닌 경우(예: 1900)는 평년으로 올바르게 구분된다.

과제 4. 사칙연산 · 거듭제곱 계산기

■ 문제 요약

사칙연산(+, -, *, /)과 거듭제곱(^)이 가능한 계산기를 작성한다.

■ 접근 방법

- 입력을 '숫자 연산자 숫자' 형식으로 받기 위해 `scanf("%f %c %f", ...)` 서식을 사용하였다. 연산자는 char형 변수로 받는다.
- 연산자에 따라 if / else if 로 분기하여 결과를 계산한다.
- 거듭제곱은 별도 연산자가 없으므로, `밑(num1)`을 `지수(num2)`만큼 반복해서 곱하는 방식으로 구현하였다.

■ 핵심 코드

```
char ao; // 연산자(부호)
scanf("%f %c %f", &num1, &ao, &num2);
```

```

if (ao == '+')      printf("%.2f + %.2f = %.2f", num1, num2, num1 + num2);
else if (ao == '-') printf("... = %.2f", num1 - num2);
else if (ao == '*') printf("... = %.2f", num1 * num2);
else if (ao == '/') printf("... = %.2f", num1 / num2);
else if (ao == '^') { // 거듭제곱은 반복 곱셈으로 계산
    float result = num1;
    for (int i = 1; i < num2; i++) result *= num1;
    printf("%.2f ^ %.2f = %.2f", num1, num2, result);
}

```

■ 사용한 개념

char 자료형, scanf 서식 문자열(%f %c %f), if / else if 조건문, for 반복문, 산술 연산자.

■ 예외처리 및 한계

- 입력 시 숫자와 연산자 사이에 공백을 두어야 %c 가 연산자를 올바르게 읽는다는 점을 주석으로 명시하였다.
- 한계: 0으로 나누는 경우와 정의되지 않은 연산자가 입력되는 경우는 별도 처리하지 않았다. 또한 거듭제곱은 반복문으로 계산하므로 양의 정수 지수에 대해서만 정확하다.

과제 5. 나비형태 별 찍기

■ 문제 요약

예제 출력의 규칙을 유추하여, 값을 입력받은 뒤 나비 형태의 별을 출력한다.

■ 접근 방법

- 한 줄을 좌측 부분과 우측 부분으로 나누어, y축(세로축) 대칭이 되도록 두 개의 내부 반복문으로 구성하였다.
- 윗부분은 1줄부터 num줄까지 늘어나고, 아랫부분은 x축(가로축) 대칭이 되도록 num-1줄부터 1줄까지 줄여가며 출력한다.

■ 핵심 코드

```

// 윗별
for (int i = 1; i <= num; i++) {
    for (int j = 1; j <= num; j++) // 왼쪽 절반
        printf(j <= i ? "*" : " ");
    for (int j = num; j > 0; j--) // y축 대칭(오른쪽 절반)
        printf(j <= i ? "*" : " ");
    printf("\n");
}
// 아랫별 : 윗별보다 1줄 적게, x축 대칭
for (int i = num-1; i >= 1; i--) { ... }

```

■ 사용한 개념

이중 for 반복문, if / else 조건문(코드에서는 삼항 형태로 요약), 관계 연산자.

■ 예외처리 및 한계

- 윗부분과 아랫부분의 줄 수 차이(아랫부분이 1줄 적음)를 반복 범위로 조절하여 위·아래가 정확히 대칭이 되도록 하였다.
- 한계: 입력값이 콘솔 폭보다 크면 줄바꿈이 흐트러질 수 있고, 0 이하 입력은 고려하지 않았다.

과제 6. 삼각형(속 빈) 별 찍기

■ 문제 요약

예제 출력의 규칙을 유추하여, 값을 입력받은 뒤 테두리만 있는 삼각형 모양(밑변은 채워짐)의 별을 출력한다.

■ 접근 방법

- 과제 5의 좌우 대칭 아이디어를 참고하되, 이번에는 '행 번호(i) + 열 번호(j) 가 num+1 이 되는 자리'에만 별을 찍어 대각선(빗변)을 만들었다.
- 같은 조건을 y축 대칭으로 한 번 더 적용해 좌·우 빗변을 완성하고, 마지막 줄은 별 2n-1개로 밑변을 채웠다.

■ 핵심 코드

```
for (int i = 1; i <= num-1; i++) {           // 마지막 줄 제외
    for (int j = 1; j <= num; j++)           // 왼쪽 빗변
        printf(j + i == num + 1 ? "*" : " ");
    for (int j = num-1; j > 0; j--)          // 오른쪽 빗변(y축 대칭)
        printf(j + i == num + 1 ? "*" : " ");
    printf("\n");
}
for (int i = 1; i < 2*num; i++) printf("*"); // 밑변 : 2n-1개
```

■ 사용한 개념

이중 for 반복문, 조건식, 산술 연산자($i + j == \text{num} + 1$).

■ 예외처리 및 한계

- 빗변의 위치를 'i + j 값'이라는 하나의 규칙으로 일반화하여, 입력값이 달라져도 삼각형 모양이 유지되도록 하였다.
- 마지막 줄만 반복 범위를 num-1 로 분리해 밑변을 별도로 채움으로써 원하는 출력 형태를 정확히 맞추었다.

과제 7. 순열 · 중복순열 · 조합 · 중복조합

■ 문제 요약

n과 r을 입력받아 순열(P), 중복순열(Π), 조합(C), 중복조합(H)을 계산한다.

■ 접근 방법

- 팩토리얼을 반복문으로 직접 계산하고, 각 공식에 맞게 분자(numerator)와 분모(denominator)를 구한 뒤 나누어 출력한다.
- 순열 $P = n! / (n-r)!$, 중복순열 $\Pi = n^r$, 조합 $C = n! / ((n-r)! \cdot r!)$ 로 계산한다.
- 중복조합 $H = C(n+r-1, r)$ 임을 이용해, n 을 $n+r-1$ 로 바꾼 뒤 조합과 동일한 계산 구조를 재사용하였다.

■ 핵심 코드

```
scanf("%d %d", &n, &r);

// 순열 P = n! / (n-r)!
for (int i = 1; i <= n; i++) numerator *= i;
for (int i = 1; i <= n-r; i++) denominator *= i;
printf("순열(P): %d\n", numerator / denominator);

// 중복순열 Π = n^r
for (int i = 1; i <= r; i++) numerator *= n;

// 조합 C = n! / ((n-r)! * r!) (분모에 r! 를 추가로 곱함)

// 중복조합 H : n = n + r - 1 로 바꿔 조합 공식 재사용
n = n + r - 1;
```

■ 사용한 개념

for 반복문을 이용한 팩토리얼 계산, 변수 초기화·재사용, 산술 연산자.

■ 예외처리 및 한계

- 각 계산 사이마다 numerator·denominator 를 1로 다시 초기화하여, 앞 계산의 값이 다음 계산에 섞이지 않도록 하였다.
- 중복조합 계산에서 n 값을 마지막에 변경하므로, 순서상 중복조합을 가장 나중에 계산하도록 배치하였다.
- 한계: 모든 계산에 int 자료형을 사용하므로 n, r 이 커지면 팩토리얼에서 오버플로우가 발생할 수 있으며, $n < r$ 과 같은 잘못된 입력은 검증하지 않았다.

마무리

이번 과제를 통해 입력(scanf)과 출력(printf)의 서식 지정, 조건문과 반복문의 조합, 그리고 반복문의 시작·종료 조건을 세밀하게 조정하는 연습을 할 수 있었다. 특히 별 찍기 과제에서는 출력 예시로부터 좌표(행·열)와 별 위치 사이의 규칙을 직접 유추하는 과정이 인상 깊었다.

반면, 정수 자료형의 오버플로우나 잘못된 입력에 대한 처리 등은 아직 보완할 부분으로 남아 있어, 앞으로 자료형의 범위와 예외 상황을 함께 고려하며 코드를 작성하려 한다.